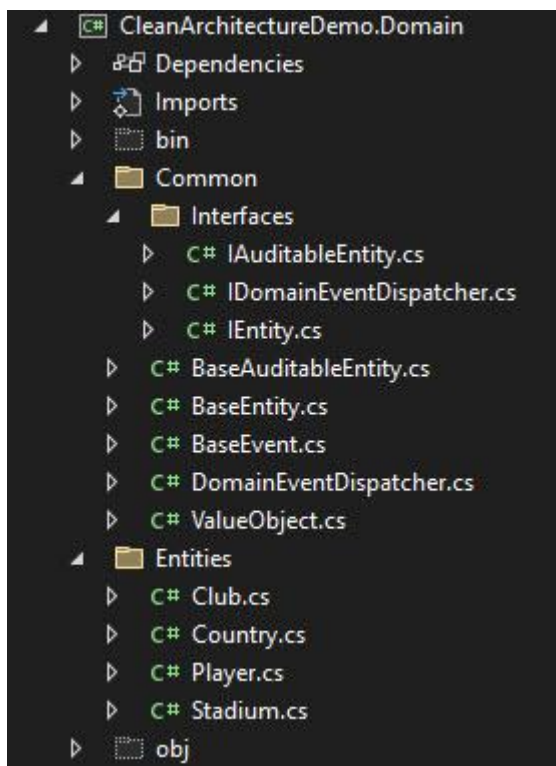


Below is a **real, clean, well-structured ASP.NET Core Web API project** that demonstrates **Separation of Concerns (SoC)** using:

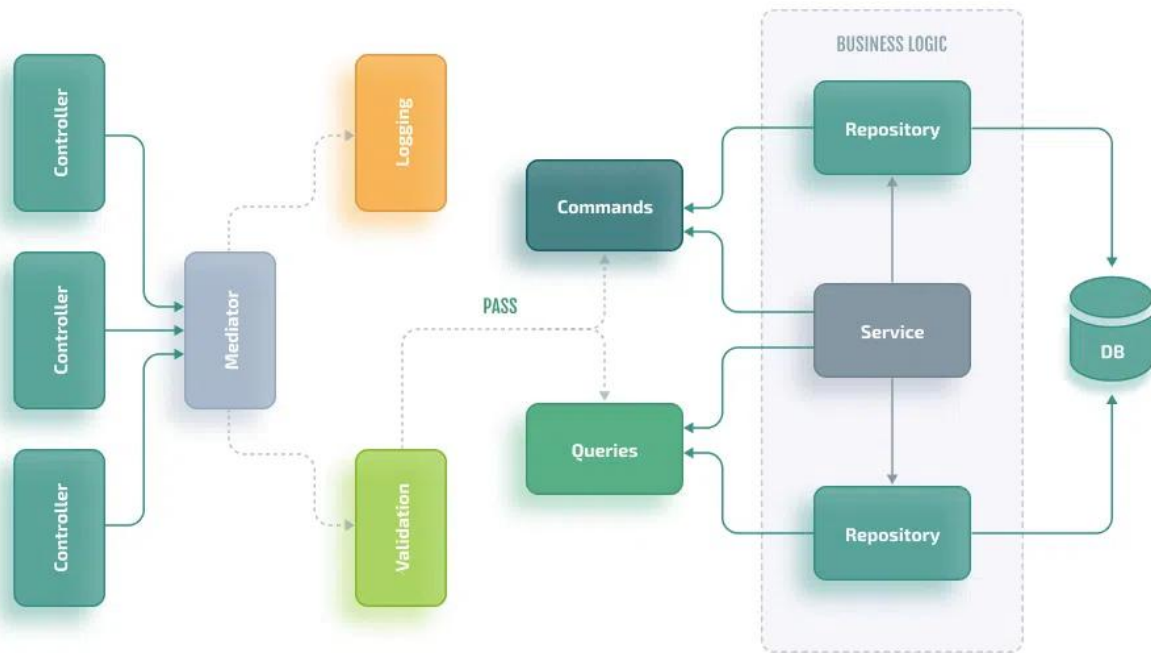
- Controllers → Request/Response only
- Services → Business logic
- Repositories → Data access
- DTOs → Data transfer
- Models → Domain entities

I'll show full folder structure + complete working code for each layer.

Project Structure (Shows Separation of Concerns Clearly)



Application architecture



www.apriorit.com

MyApp/

|

|— Controllers/

| └─ ProductsController.cs

|

|— Services/

| └─ Interfaces/

| | └─ IProductService.cs

| └─ ProductService.cs

|

|— Repositories/

| └─ Interfaces/

| | └─ IProductRepository.cs

```
|   └─ ProductRepository.cs
|
|   └─ Models/
|       └─ Product.cs
|
|       └─ DTOs/
|           └─ ProductDto.cs
|
└─ Program.cs
```

This structure is PERFECT for understanding **Separation of Concerns**.

1. Model (Domain Entity) — Only Data Structure

Models/Product.cs

```
namespace MyApp.Models
```

```
{
    public class Product
    {
        public int Id { get; set; }
        public string Name { get; set; }
        public decimal Price { get; set; }
    }
}
```

✓ Contains **only domain data**

✗ No business logic

✗ No database code

2. DTO (Data Transfer Object) — for API input/output

DTOs/ProductDto.cs

```
namespace MyApp.DTOs
```

```
{  
    public class ProductDto  
    {  
        public string Name { get; set; }  
        public decimal Price { get; set; }  
    }  
}
```

✓ Avoids exposing internal domain model

✓ Good for validation

3. Repository Interface — Defines Data Access Contract

Repositories/Interfaces/IProductRepository.cs

```
using MyApp.Models;
```

```
namespace MyApp.Repositories.Interfaces
```

```
{  
    public interface IProductRepository  
    {  
        IEnumerable<Product> GetAll();  
        Product Get(int id);  
        void Add(Product product);  
        void Delete(int id);  
    }  
}
```

```
}
```

✓ Defines **what** operations are supported

✗ Does NOT care **how** data is stored

(DB could be SQL, in-memory, MongoDB, etc.)

📄 4. Repository Implementation — Actual Data Access Logic

Repositories/ProductRepository.cs

```
using MyApp.Models;
```

```
using MyApp.Repositories.Interfaces;
```

```
namespace MyApp.Repositories
```

```
{
```

```
    public class ProductRepository : IProductRepository
```

```
    {
```

```
        private readonly List<Product> _products = new();
```

```
        public IEnumerable<Product> GetAll() => _products;
```

```
        public Product Get(int id) => _products.FirstOrDefault(x => x.Id == id);
```

```
        public void Add(Product product)
```

```
        {
```

```
            product.Id = _products.Count + 1;
```

```
            _products.Add(product);
```

```
        }
```

```

    public void Delete(int id)
    {
        var product = Get(id);
        if (product != null)
            _products.Remove(product);
    }
}

```

- ✓ Only handles storage
 - ✓ No business rules
 - ✓ No controller logic
-

5. Service Interface — Defines Business Logic Contract

Services/Interfaces/IProductService.cs

```
using MyApp.DTOs;
```

```
using MyApp.Models;
```

```
namespace MyApp.Services.Interfaces
```

```

{
    public interface IProductService
    {
        IEnumerable<Product> GetAllProducts();
        Product GetProduct(int id);
        void CreateProduct(ProductDto dto);
        void RemoveProduct(int id);
    }
}

```

```
}
```

⚙️ 6. Service Implementation — Contains Business Logic

Services/ProductService.cs

```
using MyApp.DTOs;
```

```
using MyApp.Models;
```

```
using MyApp.Repositories.Interfaces;
```

```
using MyApp.Services.Interfaces;
```

```
namespace MyApp.Services
```

```
{
```

```
    public class ProductService : IProductService
```

```
    {
```

```
        private readonly IProductRepository _repo;
```

```
        public ProductService(IProductRepository repo)
```

```
        {
```

```
            _repo = repo;
```

```
        }
```

```
        public IEnumerable<Product> GetAllProducts() => _repo.GetAll();
```

```
        public Product GetProduct(int id) => _repo.Get(id);
```

```
        public void CreateProduct(ProductDto dto)
```

```
        {
```

```

        if (dto.Price <= 0)
            throw new Exception("Price must be greater than zero.");

        var product = new Product
        {
            Name = dto.Name,
            Price = dto.Price
        };

        _repo.Add(product);
    }

    public void RemoveProduct(int id)
    {
        _repo.Delete(id);
    }
}

```

- ✓ Handles validation
- ✓ Handles business rules
- ✓ Does NOT know about HTTP
- ✓ Does NOT know about controllers

7. Controller (API Endpoint) — Handles HTTP Only

Controllers/ProductsController.cs

```
using Microsoft.AspNetCore.Mvc;
```

```
using MyApp.DTOs;

using MyApp.Services.Interfaces;


namespace MyApp.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class ProductsController : ControllerBase
    {
        private readonly IProductService _service;


        public ProductsController(IProductService service)
        {
            _service = service;
        }


        [HttpGet]
        public IActionResult GetAll()
        {
            return Ok(_service.GetAllProducts());
        }


        [HttpGet("{id}")]
        public IActionResult Get(int id)
        {
            var product = _service.GetProduct(id);
```

```

        if (product == null)
            return NotFound();

        return Ok(product);
    }

    [HttpPost]
    public IActionResult Create(ProductDto dto)
    {
        _service.CreateProduct(dto);
        return Ok("Product created");
    }

    [HttpDelete("{id}")]
    public IActionResult Delete(int id)
    {
        _service.RemoveProduct(id);
        return Ok("Product deleted");
    }
}

```

- ✓ Controls HTTP
- ✓ No business logic
- ✓ No database operations

⚡ 8. Register Services & Repositories in DI

Program.cs

```
var builder = WebApplication.CreateBuilder(args);
```

```
builder.Services.AddControllers();
```

```
// Dependency Injection
```

```
builder.Services.AddScoped<IProductRepository, ProductRepository>();
```

```
builder.Services.AddScoped<IProductService, ProductService>();
```

```
var app = builder.Build();
```

```
app.MapControllers();
```

```
app.Run();
```

How This Project Demonstrates SoC

Concern	Layer
HTTP processing	Controller
Business logic	Service
Data access	Repository
Domain model	Model
External communication	DTO

Each part focuses on **ONE responsibility** → Perfect example of Separation of Concerns.